

UNITÉ 4 · SEMAINES 5–6

Processus et mémoire

📅 Semaines 5–6 · dates à confirmer </> Laboratoire TD3 (corrigé + défi IA) ~ 7 animations

MANUEL M4 · THÉORIE DE L'UNITÉ

Processus et mémoire

Lecture de base de l'unité 4 (≈ 2 h, deux semaines). Chaque section se termine par un lien

Pratiquez →. Les sorties sont réelles: programmes du TD3 exécutés sur la machine du cours (Ubuntu 26.04, Linux 7.0, x86-64).

SOMMAIRE

- | | | | |
|----|--|----|--|
| §1 | Le processus et son espace d'adressage | §4 | <code>fork</code> , copie sur écriture et tampons |
| §2 | Mémoire virtuelle: pages, défauts et remplacement | §5 | <code>exec</code> , <code>exit</code> et <code>wait</code> |
| §3 | Le processus pour le noyau: PCB, états et ordonnancement | §6 | Le premier processus, les sessions et les services |

§1 Le processus et son espace d'adressage

Un **processus** est un programme en cours d'exécution avec son contexte: mémoire, registres, descripteurs ouverts, identifiants. Chaque processus a un **espace d'adressage privé**: les adresses qu'il utilise sont *virtuelles*, et la même adresse dans deux processus pointe vers une mémoire physique différente. Sur x86-64, les pointeurs ont 64 bits mais 48 sont utilisés: 256 Tio, la moitié basse pour le processus et la moitié haute pour le noyau.

L'espace est organisé en **zones**: le *code* (text, en lecture seule et exécutable), les *données* (globales initialisées) et le *bss* (globales non initialisées), le *tas* (petits `malloc`), les régions *mmap* (bibliothèques et grands `malloc`) et la *pile* (variables locales, qui grandit vers le bas). Le programme `zones` du TD3 affiche une adresse de chaque zone et la région de `/proc/self/maps` qui la contient:

Sortie réelle

```
$ ./zones
zone                address    perm  region
code   (main)        0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized)   0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array)   0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)    0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0))  0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)        0x71b345064080  r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
stack  (local)           0x7fff6c1e68b4  rw-p  [stack]
```

Si vous le lancez plusieurs fois, les adresses changent: le noyau rend aléatoire la base de chaque région (**ASLR**, *address space layout randomization*) pour qu'un attaquant ne puisse pas prédire où sont le code ou la pile. La glibc sert les `malloc` de 128 Kio ou plus avec un `mmap` à part, hors du tas, qui est rendu au système dès qu'il est libéré.

Pratiquez → TD3 · 5 (zones)

§2 Mémoire virtuelle: pages, défauts et remplacement

La mémoire est divisée en **pages** (4 Kio sur x86-64; aussi des grandes pages de 2 Mio et 1 Gio) et la mémoire physique en **cadres** de même taille. Une **table des pages** propre à chaque processus indique dans quel cadre se trouve chaque page et avec quelles permissions (lecture, écriture, exécution, utilisateur). Sur x86-64, la table a 4 niveaux: une adresse virtuelle est découpée en quatre indices de 9 bits et un décalage de 12 bits. Le matériel — la **MMU** — traduit à chaque accès, et la **TLB** garde en cache les traductions récentes pour ne pas parcourir la table à chaque fois.

Si l'entrée de la table n'est pas présente, la MMU lève un **défaut de page** et saute dans le noyau. Si l'adresse appartient à une région valide, le noyau alloue un cadre, le met à zéro (ou le lit depuis le disque) et relance l'instruction: c'est l'**allocation paresseuse**. Si elle n'appartient à aucune région, ou si l'accès viole les permissions (écrire dans le code, exécuter sur la pile), il envoie `SIGSEGV`. Mesuré avec `lazy` :

Sortie réelle

```
$ ./lazy 1024
start                VmRSS =    1756 KiB    page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc          VmRSS =    1952 KiB    page faults so far = 8
after touching half of the pages VmRSS = 526300 KiB    page faults so far = 131081
pages touched: 131072 (4096 bytes each) page faults: 131073    1968 ns per fault
after free            VmRSS =    2012 KiB    page faults so far = 131081
```

Linux est aussi *optimiste* (overcommit) : il promet plus de mémoire qu'il n'en a, car presque personne n'utilise tout ce qu'il réserve. S'il en manque vraiment, l'**OOM killer** termine des processus. Quand il ne reste plus de cadres libres, le noyau choisit une page **victime** et l'envoie sur le disque (swap) ou la compresse en RAM (zswap, zram). L'algorithme idéal (OPT) évincerait la page utilisée le plus tard ; les algorithmes réels approchent LRU avec le bit d'accès que positionne la MMU (l'algorithme de l'horloge ; sous Linux, listes active/inactive et MGLRU). Avec FIFO, l'**anomalie de Belady** peut se produire : plus de cadres, plus de défauts. Si l'ensemble des pages utilisées ne tient pas en mémoire, le système passe plus de temps à paginer qu'à calculer : c'est l'**écroulement** (thrashing).

Pratiquez → A4.1 mémoire virtuelle · A4.5 remplacement de pages

§3 Le processus pour le noyau : PCB, états et ordonnancement

Le noyau décrit chaque processus par un **bloc de contrôle** (PCB ; sous Linux, `struct task_struct`) : PID et PPID, propriétaire et permissions, état, registres sauvegardés, priorité, temps processeur, table des descripteurs, limites (`getrlimit`), signaux et table des pages. On peut l'inspecter dans `/proc/PID/`, que lisent `ps` et `top`.

Un processus passe par plusieurs **états** : *prêt* (attend le processeur) et *en exécution* — tous deux `R` dans `ps` —, *bloqué* en attente d'E/S (`S`, ou `D` s'il ne peut pas être interrompu), *arrêté* par `SIGSTOP` / `SIGTSTP` (`T`) et *zombie* (`Z`) entre sa fin et le `wait` de son parent.

Passer le processeur d'un processus à un autre est un **changement de contexte** : sauvegarder les registres de l'un, charger ceux de l'autre et changer de table des pages. L'**ordonnanceur** décide à qui c'est le tour. Les algorithmes classiques : FCFS (ordre d'arrivée ; souffre de l'effet de convoi), SJF et SRTF (le plus court d'abord ; ils minimisent l'attente moyenne mais exigent d'estimer les durées), Round Robin (tours d'un *quantum* ; bonne réactivité, plus de changements de contexte) et priorités (avec un risque de famine, corrigé par le vieillissement). Depuis la version 6.6 (2023), Linux utilise **EEVDF**, qui répartit le processeur selon un temps virtuel pondéré par la priorité (`nice`) et sert d'abord l'échéance la plus proche.

Pratiquez → A4.3 états · A4.4 ordonnancement

§4 `fork`, copie sur écriture et tampons

Sous Unix, un nouveau processus naît comme un **clone** d'un autre : `fork()` renvoie 0 dans l'enfant, le PID de l'enfant dans le parent et -1 en cas d'échec. L'enfant reçoit une copie de tout : mémoire, descripteurs (qui pointent vers les mêmes entrées de la table des fichiers ouverts, unité 3), gestionnaires et masque de signaux — mais pas les signaux en attente. Copier toute la mémoire coûterait très cher, alors Linux utilise la **copie sur écriture** : il ne copie que les tables des pages, marque tout en lecture seule et ne copie chaque page que lorsque l'un des deux y écrit :

Sortie réelle

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096 bytes)
fork took 6.70 ms (only the page tables are copied)
child reads 65536 pages: 0 page faults (shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults (one copy per page), 2753 ns per copy
parent: the child exited with status 0; my copy still says 'x'
```

La copie inclut aussi ce que la bibliothèque C a en attente. `printf` n'appelle pas `write` : il accumule dans un **tampon** vidé à chaque ligne (stdout vers un terminal), quand il est plein (stdout vers un tube ou un fichier), avec `fflush` ou à l'`_exit` ; stderr n'a pas de tampon et `_exit` ne vide rien. Si du texte était encore dans le tampon au moment du `fork`, le parent et l'enfant l'écrivent tous les deux : il apparaît deux fois.

Sortie réelle

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

Pratiquez → A4.2 copie sur écriture · A4.6 tampons · TD3 · 2 et 3

§5 exec, exit et wait

`execve(path, argv, envp)` remplace le programme du processus : il charge un nouveau code et de nouvelles données, crée un nouveau tas et une nouvelle pile et saute au point d'entrée. Il conserve le PID, le PPID et les descripteurs ouverts (sauf ceux marqués `O_CLOEXEC`) ; avec le bit SUID, l'UID effectif devient celui du propriétaire du fichier. S'il réussit, il ne revient pas. Les variantes de bibliothèque (`execl`, `execv`, `execvp` ...) changent la façon de passer les arguments et la recherche dans le `PATH`. Entre `fork` et `exec`, l'enfant peut réorganiser ses descripteurs : c'est ainsi que le shell implémente `>`, `<` et les tubes (le `launch` du TD3 · 4, et l'unité 5).

Un processus se termine avec `exit(status)` — qui appelle les fonctions `atexit`, vide les tampons stdio et appelle l'appel système `_exit` — ou par un signal. Le noyau ferme ses fichiers et libère sa mémoire, confie ses enfants vivants à `init` ou au *subreaper* le plus proche et prévient le parent avec `SIGCHLD`. Le PCB reste comme **zombie** jusqu'à ce que le parent appelle `wait` ou `waitpid`, qui renvoie le statut empaqueté ; les macros le décodent : `WIFEXITED` / `WEXITSTATUS`, `WIFSIGNALED` / `WTERMSIG`, `WIFSTOPPED` et `WIFCONTINUED` (avec les options `WUNTRACED` et `WCONTINUED` ; `WNOHANG` ne bloque pas).

Sortie réelle

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has called wait yet:
    PID    PPID STAT COMMAND
    356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7; now ps finds nothing:
    PID    PPID STAT COMMAND
2) orphan: my PID is 356666
   grandchild 356679: parent was 9771, now it is 9771 (systemd)
```

Sortie réelle

```
$ ./launch out.txt ./segfault
launch: ./segfault killed by signal 11 (SIGSEGV), core dumped
[exit status 139]
```

Pratiquez → A4.7 fork, exec et wait · TD3 · 4

§6 Le premier processus, les sessions et les services

Si chaque processus naît d'un `fork`, quelqu'un doit être le premier. Le **processus 0** est construit par le noyau au démarrage (un par processeur, en mode noyau) : il initialise la mémoire virtuelle, crée les threads du noyau (`kthreadd`, PID 2) et le **processus 1**, qui passe en mode utilisateur et exécute `/sbin/init` — aujourd'hui **systemd** dans presque toutes les distributions —, démarre les services et adopte les orphelins. Sur la machine du cours, `ps -p 1 -o comm=` répond `systemd` (TD0).

Une **session** regroupe des processus, typiquement ceux d'un terminal, avec un chef ; si le chef se termine, les autres reçoivent `SIGHUP`. `setsid()` crée une nouvelle session sans terminal de contrôle. Un **démon** classique se détachait du terminal avec `fork`, le parent se terminant et l'enfant appelant `setsid`. Aujourd'hui, sous Linux, les services sont lancés par `systemd` : le programme tourne au premier plan et `systemd` s'occupe de la session, des journaux (`journalctl`), des redémarrages et des limites de ressources (`cgroups`, unité 9).